

HXC-152 + HXC-151 evidence

HXC-152 + HXC-151 — captured four-quadrant evidence (§11.4.83 / §11.4.115)

Field	Value
Revision	1
Created	2026-07-29
Last modified	2026-07-29
Status	active
Items	HXC-152 (Bug/High), HXC-151 (Bug/Medium)
Fixed artifact	e879702c + the uncommitted guard conversions in this commit

Table of contents

- [What was proven](#)
- [Headline result — HXC-152](#)
- [Four-quadrant matrices](#)
- [The gastatus cleanup-deadlock finding](#)
- [Enumerated residual search](#)
- [Package verification](#)
- [Honest boundaries](#)

What was proven

Five RED/GREEN polarity guards in `helix_code/internal/server/` were made falsifiable. Method per guard (§11.4.115): reconstruct the pre-fix artifact in a throwaway git worktree off HEAD with submodules/ symlinked in (the inner module replaces to `../submodules/*`), reverting **one** production fix at a time, then run both polarities on both artifacts. Required pattern is PASS / FAIL / FAIL / PASS.

go test's own exit code was captured directly (subshell redirect to a file, then \$?), never through a pipe.

Headline result — HXC-152

llm_working_funnel_test.go defaulted to **RED** when RED_MODE was unset, so its fix-verifying GREEN assertions had almost certainly **never executed** while the test still reported PASS. The default is now GREEN via the package-shared redMode(t).

Those GREEN assertions were run for the first time and they PASS. No defect was being hidden by the inverted default — the funnel filtering works. Confirmed both with RED_MODE=0 explicitly and with the variable genuinely unset.

Four-quadrant matrices

Reverted fix per guard: systemstatus 7cc89da9, qastatus 0dfd0fbc, rag 225cdf77 (surgical — see boundaries), funnel 4d7a89ed (handlers.go only).

guard	pre-fix RED	pre-fix GREEN	fixed RED
TestGuard_GetSystemStatus_NilDB	PASS rc=0	FAIL rc=1	FAIL rc=1
TestGuard_GetQASessionStatus_RecursiveRLockDeadlock	PASS rc=0 (8.07s)	FAIL rc=1	FAIL rc=1
TestGenerateLLM_RAG_Enabled_Augments_RegressionGuard	PASS rc=0	FAIL rc=1	FAIL rc=1
TestServerListLLMModels_WorkingFunnelEndToEnd	PASS rc=0	FAIL rc=1	FAIL rc=1
TestServerListLLMProviders_WorkingFunnelEndToEnd	PASS rc=0	FAIL rc=1	FAIL rc=1

All five match. Selected verbatim observables proving each cell fired on the intended assertion rather than incidentally:

- systemstatus, pre-fix GREEN: getSystemStatus panicked on a nil-db Server: runtime error: invalid memory address or nil pointer dereference

- systemstatus, fixed RED: RED_MODE: expected the real getSystemStatus to panic ... got none – the defect did not reproduce
- rag, pre-fix GREEN: "what is the HelixCode server binary called?" does not contain "The HelixCode server binary is bin/helixcode." **and** "[]" should have 1 item(s), but has 0—prompt unaugmented *and* retriever never called
- rag, fixed RED: provider actually received Context retrieved from the local knowledge base:\n\n[1] The HelixCode server binary is bin/helixcode.\n(source: docs/build.md)\n\n--\n\nwhat is ...
- funnel models, pre-fix GREEN: map[claude-failed:true claude-good:true claude-lowscore:true claude-pending:true gpt-good:true] should have 1 item(s), but has 5—the pre-fix handler really served the whole unfiltered catalog over HTTP
- funnel models, fixed RED: map[claude-good:true] should have 5 item(s), but has 1
- funnel providers, fixed RED: fails at :184 RED: the pre-fix handler lists openai despite NO openai key being present — proving the branch that previously only t.Skip()ed is now live
- qastatus, pre-fix RED: child probe printed QASTATUS_PROBE_WEDGED accept="" attempt=1, exit 97
- qastatus, fixed RED: child probe printed QASTATUS_PROBE_SURVIVED attempts=40, exit 0

The qastatus cleanup-deadlock finding

Raw log: [qastatus_inprocess_cleanup_deadlock_prefix RED1.txt](#)

The first conversion of this guard drove the real handler **in-process**. On a pre-fix artifact it could never report PASS, so its top-left quadrant was unreachable and the guard was not falsifiable. Measured with 0dfd0fbc reverted:

```
handlers_qastatus_deadlock_guard_test.go:160: RED: real getQASessionStatus
panic: test timed out after 1m30s
...
sync.(*RWMutex).Lock(...)
dev.helix.code/internal/helixqa.(*Engine).Shutdown(...) wrapper.go:210
dev.helix.code/internal/server.setupQATestServer.func1() qa_handlers_test
testing.(*common).Cleanup.func1()
```

and concurrently the wedged handler goroutine:

```
goroutine 98 [sync.RWMutex.RLock]:
dev.helix.code/internal/helixqa.(*SessionState).MarshalJSON(...) wrapper.
```

```
encoding/json.Marshal(...)
dev.helix.code/internal/server.(*Server).getQASessionStatus(...)
```

The defect **did** reproduce, and the run **still** exited 1 on the go-test timeout: setupQATestServer registers t.Cleanup(qaEngine.Shutdown), and Shutdown takes s.Mu.Lock(), which blocks forever behind the wedged reader.

Fix: both polarities now observe the handler from a **child probe process** that calls os.Exit(97) on wedge, bypassing t.Cleanup entirely. This also makes a future regression fail the standing GREEN guard **cleanly in ~8s** instead of hanging to a suite timeout — measured pre-fix GREEN: rc=1 in 8.03s with Exit 97 means the recursive-RLock self-deadlock has regressed.

Enumerated residual search

Not “no others found” (§11.4.118). Every polarity switch in the package was enumerated by **every** env-var name in use, not just RED_MODE:

file	switch	default
handlers_systemstatus_guard_test.go	HELIX_RED_MODE	GREEN
handlers_qastatus_deadlock_guard_test.go	HELIX_QASTATUS_RED_MODE	GREEN
llm_generate_regression_test.go (redMode)	RED_MODE	GREEN
server_nil_redis_test.go (×2)	RED_MODE	GREEN
llm_auth_guard_test.go	RED_LLM_AUTH	GREEN
ws_auth_test.go	RED_WS_AUTH	GREEN
wire_facade_auth_test.go	RED_WIRE_FACADE_AUTH	GREEN
handlers_project_idor_test.go	RED_PROJECT_IDOR	GREEN

No inverted default remains. Grepping Replica|replicat|preFix|old[A-Z]...(across internal/server/*_test.go returns empty, and each named replica symbol is gone: preFixSystemStatusHealthCheck, reproduceRecursiveRLockDeadlock, funnelRedMode, oldStreamPumpReplica, oldResolveDefaultModelReplica — 0 occurrences each.

Package verification

On the fixed artifact, all five files in place:

```
go vet    ./internal/server/...      rc=0
go test   -count=1 ./internal/server/...  rc=0   ok 1.168s
```

```
go test -race -count=1 ./internal/server/... ok 3.311s
go build ./internal/... ./cmd/... rc=0
gofmt -l <the five touched files> empty
```

go build ./... fails at pristine HEAD on missing X11/GLFW headers — a pre-existing host gap, not a defect of this change (§11.4.201).

Honest boundaries

1. **RAG pre-fix is a narrowed counterfactual.** A whole-commit revert of 225cdf77 does not compile: later commits added `applyRAGContext` call sites in `wire_facade.go:651,706` that the revert orphans. The measured pre-fix shape removes only the `applyRAGContext` call inside `generateLLM` — the exact wiring this guard drives. The helper, the `streamLLM` call site and the wire-facade call sites remain. This is precise for the guard but narrower than the full commit.
2. **One intermittent flake observed, cause UNCONFIRMED.** One full-package `-race` run failed in `TestStartQASession_Success` (expected "pending", actual "running") — a **pre-existing test this work does not touch**. Mechanism established from source: `startQASession` passes `c.JSON` the same live `*SessionState` pointer the orchestrator goroutine is concurrently flipping `pending` → `running`, so the serialized body races. Repeat runs after the observation: current tree **11 clean / 1 fail**, pristine HEAD **4/4 clean**. One failure in twelve is too rare to separate “pre-existing latent flake” from “pre-existing latent flake whose rate the new child-process spawn nudges upward”, so the contribution is recorded as UNCONFIRMED rather than dismissed. The underlying racy assertion is a genuine §11.4.50 determinism defect and is **reported for its own tracked item** — deliberately not patched here, since fixing an unrelated test without its own RED-first cycle would violate §11.4.43 / §11.4.146.
3. **Not measured:** `-race` was not run against each reconstructed pre-fix artifact, only against the fixed one; and the rag guard’s sibling `stream-path/` unit tests were not measured under either polarity (out of the named guards’ scope).